Actividad 4.2 - Estándares de Codificación

María Virginia Mendizábal Miranda - A01796588

Repositorio con tres programas desarrollados en Python que implementan algoritmos básicos sin depender de funciones integradas del lenguaje. Todos los programas siguen el estándar de codificación **PEP-8** y fueron validados con **pylint** obteniendo una calificación de **10.00/10**.

Estructura del repositorio

```
actividad-4-2-coding-standards/
├── exercise1_compute_statistics/
│   ├── computeStatistics.py
│   ├── data/               (TC1.txt … TC7.txt)
│   └── evidence/           (capturas de pantalla)
├── exercise2_converter/
│   ├── convertNumbers.py
│   ├── data/               (TC1.txt … TC4.txt)
│   └── evidence/           (capturas de pantalla)
├── exercise3_word_count/
│   ├── wordCount.py
│   ├── data/               (TC1.txt … TC5.txt)
│   └── evidence/           (capturas de pantalla)
└── README.md
```

Ejecución

Cada programa se ejecuta desde la línea de comandos recibiendo como argumento un archivo de texto con los datos de entrada:

```
python computeStatistics.py fileWithData.txt
python convertNumbers.py fileWithData.txt
python wordCount.py fileWithData.txt
```

Los resultados se imprimen en consola y se guardan automáticamente en un archivo de resultados dentro de la misma carpeta de ejecución.

Ejercicio 1 - Compute Statistics (`computeStatistics.py`)

Descripción

Este programa lee un archivo de texto que contiene valores numéricos (uno por línea) y calcula las siguientes estadísticas descriptivas:

Estadística	Algoritmo utilizado
Count	Conteo de elementos válidos
Mean	Suma total / cantidad de elementos
Median	Valor central de la lista ordenada con Merge Sort
Mode	Valor con mayor frecuencia (o `#N/A` si no hay repetidos)
Variance	Varianza poblacional (divide entre *n*)
Standard Deviation	Raíz cuadrada de la varianza usando Newton-Raphson

Todas las operaciones se realizan con algoritmos propios; no se utilizan librerías como `statistics` ni funciones integradas como `sorted()` .

Los datos inválidos se reportan como error en consola y se omiten del cálculo. El archivo de salida generado es `StatisticsResults.txt` .

Casos de prueba ejecutados

Se ejecutaron **7 casos de prueba** (TC1 a TC7) que cubren distintos escenarios: conjuntos de datos pequeños, grandes, con valores negativos, decimales, datos inválidos mezclados, entre otros.

Evidencias

Evidencia 1 - Ejecución de los casos de prueba TC1 a TC5. Se observan los resultados de cada ejecución mostrando las estadísticas calculadas (COUNT, MEAN, MEDIAN, MODE, STANDARD_DEVIATION, VARIANCE y ELAPSED_TIME_SECONDS). En el caso TC5 se demuestra el manejo de datos inválidos, donde el programa reporta las líneas con errores y continúa el procesamiento con los valores válidos.

```
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise1_compute_statistics> python computeStatistics.py data\TC1.txt
COUNT: 400
MEAN: 242.32
MEDIAN: 239.5
MODE: 393.0
STANDARD_DEVIATION: 145.25810683056557
VARIANCE: 21099.917599999997
ELAPSED_TIME_SECONDS: 0.001565000423192978

PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise1_compute_statistics> python computeStatistics.py data\TC2.txt
COUNT: 1977
MEAN: 250.7840161861406
MEDIAN: 247.0
MODE: 230.0
STANDARD_DEVIATION: 144.17131868884059
VARIANCE: 20785.369132479238
ELAPSED_TIME_SECONDS: 0.002943599945865571 5

PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise1_compute_statistics> python computeStatistics.py data\TC3.txt
COUNT: 12624
MEAN: 249.77621989860583
MEDIAN: 249.0
MODE: 94.0
STANDARD_DEVIATION: 145.31784980917962
VARIANCE: 21117.27747316329
ELAPSED_TIME_SECONDS: 0.033441100036725 4

PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise1_compute_statistics> python computeStatistics.py data\TC4.txt
>>
COUNT: 12624
MEAN: 149.00267347908746
MEDIAN: 147.75
MODE: 123.75
STANDARD_DEVIATION: 130.41441961308894
VARIANCE: 17007.920843018837
ELAPSED_TIME_SECONDS: 0.041164300055243075

PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise1_compute_statistics> python computeStatistics.py data\TC5.txt
[ERROR] Line 5: invalid data -> 'ABA'
[ERROR] Line 155: invalid data -> '23,45'
[ERROR] Line 232: invalid data -> '11;54'
[ERROR] Line 239: invalid data -> '11'
COUNT: 307
```

Evidencia 2 - Ejecución de los casos de prueba TC6 y TC7, que incluyen conjuntos de datos más grandes con notación científica. Además, se muestran las dos ejecuciones de **pylint** sobre el archivo `computeStatistics.py`, obteniendo en ambas una calificación perfecta de **10.00/10**, lo que confirma el cumplimiento total del estándar PEP-8.

```
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise1_compute_statistics> python computeStatistics.py data\TC6.txt
COUNT: 3000
MEAN: 1.8790659927977473e+20
MEDIAN: 1.8800004996654 3e+20
MODE: #N/A
STANDARD_DEVIATION: 1.0738991852412603e+31
VARIANCE: 1.1530904699530647e+40
ELAPSED_TIME_SECONDS: 0.004673299961723387

PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise1_compute_statistics> python computeStatistics.py data\TC7.txt
[ERROR] Line 183: invalid data -> 'ABBA'
[ERROR] Line 229: invalid data -> 'ERROR'
COUNT: 12767
MEAN: 2.474673954997149e+20
MEDIAN: 2.4664097307429e+20
MODE: #N/A
STANDARD_DEVIATION: 1.9474693711042855e+31
VARIANCE: 2.0910793147136484e+40
ELAPSED_TIME_SECONDS: 0.02134560002014041

PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise1_compute_statistics> python -m pylint computeStatistics.py
>>

------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise1_compute_statistics> python -m pylint computeStatistics.py
>>

------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise1_compute_statistics>
```

Ejercicio 2 - Convert Numbers (`convertNumbers.py`)

Descripción

Este programa lee un archivo de texto con números enteros (uno por línea) y los convierte a sus representaciones en **binario** y **hexadecimal** utilizando algoritmos básicos de división sucesiva.

Conversión	Detalle
Binario	División sucesiva entre 2. Para negativos se usa complemento a dos de 10 bits.
Hexadecimal	División sucesiva entre 16. Para negativos se usa complemento a dos de 40 bits.

No se utilizan funciones integradas como `bin()` , `hex()` o `format()` . Los datos inválidos (texto, valores vacíos, etc.) se reportan en consola y se omiten de la tabla de resultados. El archivo de salida generado es `ConvertionResults.txt` .

Casos de prueba ejecutados

Se ejecutaron **4 casos de prueba** (TC1 a TC4) que incluyen números positivos, negativos, valores con signo explícito (`+` / `-`), datos inválidos y conjuntos de datos de distintos tamaños (desde pocos elementos hasta 200 registros).

Evidencias

Evidencia 1 - Fragmento final de la ejecución del caso de prueba TC1 (200 elementos). Se muestra la tabla con las columnas ITEM, DECIMAL, BINARY y HEXADECIMAL, junto con el tiempo de ejecución al final.

```
174     5325800 10100010100001111101000 5143E8
175     3822527 11101001010011101111111  3A53BF
176     5503858 10100111111101101110010 53FB72
177     9214055 100011001001100001100111        8C9867
178     6521769 11000111000001110101001 6383A9
179     7923796 11110001110100001010100 78E854
180     5250236 10100000001110010111100 501CBC
181     1083154 100001000011100010010   108712
182     472141  1110011010001001101     7344D
183     9597454 100100100111001000001110        92720E
184     1581679 110000010001001101111   18226F
185     656751  10100000010101101111    A056F
186     345464  1010100010101111000     54578
187     4281218 10000010101001110000010 415382
188     6558883 11001000001010010100011 6414A3
189     3852986 11101011001010101111010  3ACABA
190     6263187 10111111001000110010011 5F9193
191     5828308 10110001110111011010100 58EED4
192     8058535 11110101111011010100111 7AF6A7
193     9035191 100010011101110110110111        89DDB7
194     7922103 11110001110000110110111 78E1B7
195     9366003 100011101110100111110011        8EE9F3
196     4555717 10001011000001111000101 4583C5
197     3526753 11010111010000011000001  35D061
198     3176815 11000001111001011011111  30796F
199     858440  11010001100101001000    D1948
200     2250854 10001001011000011001 10  225866
ELAPSED_TIME_SECONDS: 0.000723700039088726
```

Evidencia 2 - Fragmento final de la ejecución del caso de prueba TC2 (200 elementos). Se puede observar la conversión correcta de números enteros grandes a binario y hexadecimal.

```
176     3101270 1011110101001001010110  2F5256
177     1382825 10101000110011010100 1   1519A9
178     9873376 100101101010011111100000        96A7E0
179     6873500 11010001110000110011100 68E19C
180     8185634 11111001110011100100010 7CE722
181     2464786 1001011001110000010010  259C12
182     8387154 11111111111101001010010 7FFA52
183     5581444 10101010010101010000100 552A84
184     8097911 11110111001000001110111 7B9077
185     8882527 100001111000100101011111        87895F
186     8941444 100010000110111110000100        886F84
187     4942703 10010110110101101101111 4B6B6F
188     101144  11000101100011000       18B18
189     7471180 11100100000000001001100 72004C
190     1932131 1110101111101101100011  1D7B63
191     8052752 11110101110000000010000 7AE010
192     6359493 11000010000100111000101 6109C5
193     1967646 111100000011000011110   1E061E
194     6575052 11001000101001111001100 6453CC
195     2323342 1000110111001110001110  23738E
196     6735760 11001101100011110010000 66C790
197     8895858 100001111011110101110010        87BD72
198     4238091 10000001010101100001011 40AB0B
199     7093069 11011000011101101001101 6C3B4D
200     39      100111  27
ELAPSED_TIME_SECONDS: 0.0007757999701425433
```

Evidencia 3 - Ejecución del caso de prueba TC3, que contiene números positivos y negativos. Se puede apreciar el correcto funcionamiento del complemento a dos tanto en binario (10 bits) como en hexadecimal (40 bits) para los valores negativos.

```
179     18      10010   12
180     33      100001  21
181     -15     1111110001      FFFFFFFFF1
182     17      10001   11
183     25      11001   19
184     -25     1111100111      FFFFFFFFE7
185     -5      1111111011      FFFFFFFFFB
186     33      100001  21
187     -13     1111110011      FFFFFFFFF3
188     33      100001  21
189     -10     1111110110      FFFFFFFFF6
190     47      101111  2F
191     47      101111  2F
192     -13     1111110011      FFFFFFFFF3
193     -32     1111100000      FFFFFFFFE0
194     1       1       1
195     1       1       1
196     -25     1111100111      FFFFFFFFE7
197     -33     1111011111      FFFFFFFFDF
198     16      10000   10
199     17      10001   11
200     4       100     4
ELAPSED_TIME_SECONDS: 0.00053759990260005
```

Evidencia 4 - Ejecución del caso de prueba TC4, que incluye datos inválidos mezclados con datos válidos. Se observan los mensajes de error para las líneas con datos no numéricos (como `'ABC'`, `'ERROR'` y `'VAL'`) y cómo el programa continúa procesando correctamente el resto de los datos, incluyendo conversiones de números negativos con complemento a dos.

```
vidad-4-2-coding-standards\exercise2_converter> python convertNumbers.py data\TC3.txt
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Prue
vidad-4-2-coding-standards\exercise2_converter> python convertNumbers.py data\TC4.txt
FILE: data\TC4.txt
ITEM    DECIMAL BINARY  HEXADECIMAL
1       -39     1111011001      FFFFFFFFD9
2       -36     1111011100      FFFFFFFFDC
3       8       1000    8
4       34      100010  22
5       17      10001   11
6       49      110001  31
7       5       101     5
[ERROR] Line 8: invalid data -> 'ABC'
8       0       0       0
9       33      100001  21
10      12      1100    C
11      -6      1111111010      FFFFFFFFFA
12      27      11011   1B
13      -4      1111111100      FFFFFFFFFC
14      -38     1111011010      FFFFFFFFDA
15      26      11010   1A
16      49      110001  31
17      29      11101   1D
18      42      101010  2A
19      -16     1111110000      FFFFFFFFF0
[ERROR] Line 21: invalid data -> 'ERR'
20      34      100010  22
21      20      10100   14
22      0       0       0
23      25      11001   19
24      45      101101  2D
25      3       11      3
26      -46     1111010010      FFFFFFFFD2
27      -46     1111010010      FFFFFFFFD2
28      29      11101   1D
29      33      100001  21
30      29      11101   1D
31      26      11010   1A
32      -5      1111111011      FFFFFFFFFB
33      -36     1111011100      FFFFFFFFDC
34      12      1100    C
35      45      101101  2D
36      -50     1111001110      FFFFFFFFCE
37      0       0       0
38      -6      1111111010      FFFFFFFFFA
[ERROR] Line 41: invalid data -> 'VAL'
ELAPSED_TIME_SECONDS: 0.0002973000518977642
```

Evidencia 5 - Ejecución de **pylint** sobre el archivo `convertNumbers.py`, obteniendo una calificación perfecta de **10.00/10**, confirmando que el código cumple completamente con el estándar PEP-8.

```
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise2_converter> python -m pylint convertNumbers.py
>>

------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Ejercicio 3 - Word Count (`wordCount.py`)

Descripción

Este programa lee un archivo de texto y cuenta la frecuencia de cada palabra distinta. Los resultados se presentan ordenados de mayor a menor frecuencia utilizando un algoritmo de **Insertion Sort** implementado manualmente.

Funcionalidad	Detalle
Lectura de palabras	Se separan por espacios en blanco, línea por línea
Conteo de frecuencia	Diccionario que acumula las apariciones de cada palabra
Ordenamiento	Insertion Sort descendente por frecuencia
Salida	Tabla con cada palabra y su frecuencia, total general y tiempo de ejecución

No se utilizan funciones de ordenamiento integradas como `sorted()` ni `list.sort()` . El archivo de salida generado es `WordCountResults.txt` .

Casos de prueba ejecutados

Se ejecutaron **5 casos de prueba** (TC1 a TC5) con archivos de texto de distintos tamaños: 100, 104, 500, 1000 y 5000 palabras respectivamente.

Evidencias

Evidencia 1 - Ejecución del caso de prueba TC1 (100 palabras). Se muestra el listado de palabras con su frecuencia y el total general (Grand Total: 100), junto con el tiempo de ejecución.

```
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise3_word_count> python wordCount.py data\TC1.txt
customized      1
oakland 1
louis   1
tab     1
consistent      1
enhanced        1
liable  1
ebony   1
wan     1
pubmed  1
math    1
tea     1
craps   1
gothic  1
permissions     1
recorded        1
cgi     1
confirm 1
hyundai 1
exhaust 1
malpractice     1
pens    1
potentially     1
glenn   1
scoring 1
andrews 1
assessed        1
adventures      1
meals   1
mortality       1
club    1
mon     1
comm    1
blues   1
collect 1
lies    1
seats   1
worse   1
guestbook       1
influences      1
kodak   1
significance    1
coastal 1
Grand Total     100
Elapsed Time: 0.000232 seconds

Results written to WordCountResults.txt
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise3_word_count>
```

Evidencia 2 - Ejecución del caso de prueba TC2 (104 palabras). Se observa el conteo correcto de frecuencias para un conjunto de datos similar al anterior.

```
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\VMA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise3_word_count> python wordCount.py data\TC2.txt
>>
night   1
tiger   1
unity   1
disposal        1
cannon  1
ou      1
hall    1
atomic  1
chosen  1
violence        1
sectors 1
ignored 1
cheese  1
frozen  1
corp    1
undo    1
tba     1
way     1
filter  1
win     1
attending       1
returned        1
anchor  1
sides   1
defence 1
biz     1
form    1
feeding 1
rentals 1
practical       1
steve   1
serious 1
mens    1
faculty 1
initiative      1
variety 1
ion     1
interface       1
remained        1
icons   1
excessive       1
jon     1
Grand Total     184
Elapsed Time: 0.000251 seconds

Results written to WordCountResults.txt
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\VMA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise3_word_count>
```

Evidencia 3 - Ejecución del caso de prueba TC3 (500 palabras). El programa procesa correctamente un volumen intermedio de datos y muestra los resultados ordenados por frecuencia descendente.

```
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise3_word_count> python wordCount.py data\TC3.txt
wal     1
reply   1
examples        1
strengthening   1
providing       1
victory 1
commented       1
used    1
debian  1
salaries        1
buildings       1
utils   1
pmc     1
manually        1
purchases       1
insert  1
improving       1
devil   1
dodge   1
cables  1
std     1
create  1
nipples 1
jane    1
holmes  1
tea     1
butler  1
motels  1
newfoundland    1
cutting 1
clocks  1
infrared        1
cowboy  1
hormone 1
windsor 1
census  1
uc      1
declined        1
index   1
equipped        1
src     1
gradually       1
placing 1
Grand Total     500
Elapsed Time: 0.000471 seconds

Results written to WordCountResults.txt
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise3_word_count>
```

Evidencia 4 - Ejecución del caso de prueba TC4 (1000 palabras). Se demuestra que el programa maneja sin problema conjuntos de datos más grandes, manteniendo el conteo y ordenamiento correcto.

```
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise3_word_count> python wordCount.py data\TC4.txt
foods   1
ft      1
dating  1
building        1
estonia 1
performs        1
equilibrium     1
sen     1
tahoe   1
disclose        1
slip    1
midlands        1
shemales        1
walking 1
racks   1
innocent        1
campaigns       1
christina       1
bulletin        1
synthesis       1
so      1
commerce        1
offensive       1
updating        1
interaction     1
received        1
ma      1
hdtv    1
rates   1
powerful        1
was     1
computing       1
cells   1
contained       1
replaced        1
calling 1
mailman 1
extent  1
corpus  1
sv      1
wishlist        1
expired 1
circular        1
Grand Total     1000
Elapsed Time: 0.001317 seconds

Results written to WordCountResults.txt
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise3_word_count>
```

Evidencia 5 - Ejecución del caso de prueba TC5 (5000 palabras). Este es el caso de prueba más grande, y el programa completa el procesamiento y genera los resultados correctamente.

```
state   1
lenses  1
sig     1
quoted  1
words   1
trash   1
seq     1
oriental        1
veterinary      1
scientific      1
stadium 1
stability       1
specialty       1
minneapolis     1
msn     1
reception       1
recipe  1
spectacular     1
regulated       1
richards        1
moved   1
minute  1
peak    1
mods    1
philosophy      1
lc      1
throw   1
lifestyle       1
whats   1
microsoft       1
tom     1
proc    1
nano    1
letters 1
sri     1
tests   1
proceeds        1
seo     1
pounds  1
suggesting      1
texas   1
postposted      1
realty  1
vaccine 1
relocation      1
Grand Total     5000
Elapsed Time: 0.062968 seconds

Results written to WordCountResults.txt
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise3_word_count>
```

Evidencia 6 - Ejecución de **pylint** sobre el archivo `wordCount.py` , obteniendo una calificación perfecta de **10.00/10**, confirmando que el código cumple completamente con el estándar PEP-8.

```
PS C:\Users\ThinkBig\Documents\A Nuestra Empresa\ACoorporativo\A VMM\MNA Tec IA A\06 Pruebas de software y aseguramiento de la calidad\04 semana\acti
vidad-4-2-coding-standards\exercise3_word_count> python -m pylint wordCount.py
>>

------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Cumplimiento del estándar PEP-8

Los tres programas fueron analizados con **pylint** y obtuvieron la calificación máxima de **10.00/10**:

Programa	Calificación pylint
`computeStatistics.py`	10.00/10
`convertNumbers.py`	10.00/10
`wordCount.py`	10.00/10

Esto garantiza que el código cumple con las convenciones de estilo de Python definidas en PEP-8, incluyendo: nombres de variables y funciones en `snake_case`, docstrings descriptivos, manejo adecuado de imports, longitud de línea controlada y estructura limpia del código.